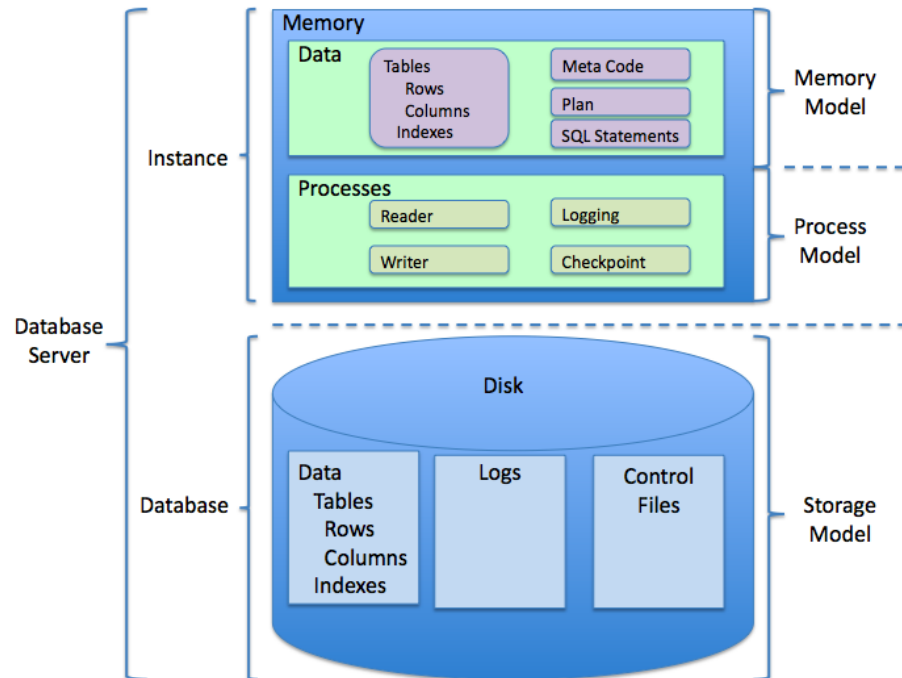


# Database Management System



By Assistant Professor Malik M Ali

# The Database Life Cycle (DBLC)

- Database Initial Study:
  - Analyze company situation
  - Define problems and constraints
  - Define objectives
  - Define scope and boundaries
- Database design
  - Create conceptual design
  - Data modal selection and DBMS selection
  - Create logical design
  - Create physical design

# The Database Life Cycle (DBLC)

- Implementation:
  - Install DBMS application
  - Create database
  - Load data
- Test and Evaluation
  - Test the database
  - Fine-tune the database
  - Evaluate the database and its application programs

# The Database Life Cycle (DBLC)

- Operations
  - Produce the required information flow
- Maintenance and evolution
  - Introduce changes
  - Make enhancements

# Types of DBMS

Malik M Ali

- DBMS can be classified by
    - ❑ The number of users supported – single / multi user
    - ❑ Where the data are located – centralized / distributed
    - ❑ The type of data stored – general purpose / discipline specific
    - ❑ Data model
-

# Types of DBMS

Malik M Ali

## ■ Single User

- This is the database environment which supports only one user accessing the database at a specific time. The DBMS might have a number of users but at a certain time only one user can log into the database system and use it. This type of DBMS systems are also called Desktop Database systems

## ■ Multi User

- This is the type of DBMS which can support a number of users simultaneously
-

# Types of DBMS

Malik M Ali

## ■ Centralized

- A database management system that supports database located at a single site is called a centralized database management system.

## ■ Distributed

- A database management system that supports database distributed across several different sites is called a distributed database management system
-

# Types of DBMS

Malik M Ali

- General Purpose DBMS:
    - A DBMS that supports a wide verity of data used in multiple disciplines
  - Discipline specific DBMS:
    - A DBMS that supports data focused on specific subject areas. When performance is a primary consideration, a special-purpose DBMS can be designed and built for a specific application.
-



# Data Model

- A model is an abstraction of a more complex real world object or event. It helps you understand the complexities of real world environment.
- Data model is a simple representation of more complex real world data structures. It is a communication tool.
- Data modeling refers to the process of creating a specific data model for a determined problem domain.

# Importance of Data Models

Are a communication tool

Give an overall view of the database

Organize data for various users

Are an abstraction for the creation of good database

# Types of DBMS with respect to data models

- Data Model: we can classify databases on the basis of data models as well.
    - **Relational data model** represents a database as a collection of tables, where each table can be stored as a separate file. Most relational databases use the high-level query language called SQL for data access and manipulation
    - **Object data model** represents a database in terms of objects, their properties and their operations. Objects with the same structure and behavior belong to a **class** and operations of each class are specified in terms of predefined procedures called methods
-

# Types of DBMS with respect to data models

- **Entity Relationship Model:**
  - Represents the entities in a system and their relationships.
  - Key concepts: Entities, attributes, relationships.
  - Used for designing and visualizing database structures.
- **Graph Databases:**
  - Use Case: Social Networks, Fraud Detection
  - Graph databases, such as Neo4j and Amazon Neptune, are optimized for managing and querying data with complex relationships. They are well-suited for scenarios where understanding and traversing relationships between entities are critical.

# Types of DBMS with respect to data models

- **Document Stores:**

- Use Case: Content Management Systems, Blogs
- Document-oriented databases, like MongoDB and CouchDB, store data in flexible, JSON-like documents. This is useful for applications where the structure of data can evolve over time, and each record can have different fields.

- **Key-Value Stores:**

- Use Case: Caching, Session Storage
- Key-value stores, such as Redis and DynamoDB, are efficient for simple data retrieval based on unique keys. They are commonly used for caching, session storage, and scenarios where quick key-based access is crucial.

# Business Rules

Brief, precise, and unambiguous description of a policy, procedure, or principle

Enable defining the basic building blocks

Describe main and distinguishing characteristics of the data

# Example of Business Rules

Examples:

1. A customer may generate many invoices
2. An invoice is generated by only one customer
3. A training session cannot be scheduled for fewer than 10 employee or more than 30 employees

**Entities:** CUSTOMER, INVOICE, EMPLOYEE, TRAINING

**Relationships:** 1:M between CUSTOMER and INVOICE, M:N between EMPLOYEE and TRAINING **Constraint:** No fewer than 10, no more than 30 employees in training

# Relational Data Model

- **Relational Model:**

- Introduced by E. F. Codd (of IBM) in 1970
- Major breakthrough, a database revolution
- Based on mathematical concept “relation”
- Relation = matrix (table) consisting of tuples (rows) and attributes (columns)
- Tables are related through a common attribute.(Fig 2.1)
- Relational Database Management Systems (RDBMS)
- Used Relational Diagrams (Fig 2.2)
- Introduced Structured Query Language (SQL)



**FIGURE  
2.1**

## Linking relational tables

Table name: AGENT (first six attributes)

Database name: Ch02\_InsureCo

| AGENT_CODE | AGENT_LNAME | AGENT_FNAME | AGENT_INITIAL | AGENT_AREACODE | AGENT_PHONE |
|------------|-------------|-------------|---------------|----------------|-------------|
| 501        | Alby        | Alex        | B             | 713            | 228-1249    |
| 502        | Hahn        | Leah        | F             | 615            | 882-1244    |
| 503        | Okon        | John        | T             | 615            | 123-5589    |

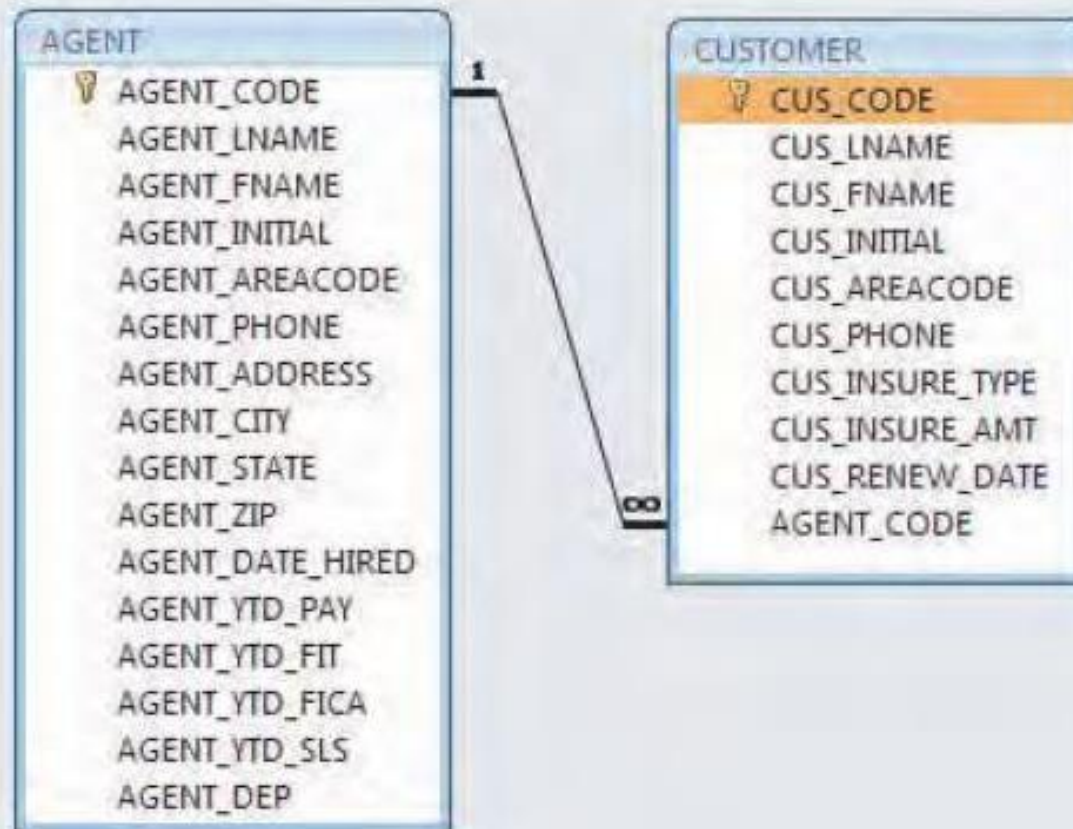
Link through AGENT\_CODE

Table name: CUSTOMER

| CUS_CODE | CUS_LNAME | CUS_FNAME | CUS_INITIAL | CUS_AREACODE | CUS_PHONE | CUS_INSURE_TYPE | CUS_INSURE_AMT | CUS_RENEW_DATE | AGENT_CODE |
|----------|-----------|-----------|-------------|--------------|-----------|-----------------|----------------|----------------|------------|
| 10010    | Ramas     | Alfred    | A           | 615          | 844-2573  | T1              | 100.00         | 05-Apr-2010    | 502        |
| 10011    | Dunne     | Leona     | K           | 713          | 894-1238  | T1              | 250.00         | 16-Jun-2010    | 501        |
| 10012    | Smith     | Kathy     | W           | 615          | 894-2285  | S2              | 150.00         | 29-Jan-2011    | 502        |
| 10013    | Olowski   | Paul      | F           | 615          | 894-2180  | S1              | 300.00         | 14-Oct-2010    | 502        |
| 10014    | Orlando   | Myron     |             | 615          | 222-1672  | T1              | 100.00         | 28-Dec-2010    | 501        |
| 10015    | O'Brian   | Amy       | B           | 713          | 442-3381  | T2              | 850.00         | 22-Sep-2010    | 503        |
| 10016    | Brown     | James     | G           | 615          | 297-1228  | S1              | 120.00         | 25-Mar-2011    | 502        |
| 10017    | Williams  | George    |             | 615          | 290-2556  | S1              | 250.00         | 17-Jul-2010    | 503        |
| 10018    | Farriss   | Anne      | G           | 713          | 382-7185  | T2              | 100.00         | 03-Dec-2010    | 501        |
| 10019    | Smith     | Olette    | K           | 615          | 297-3809  | S2              | 500.00         | 14-Mar-2011    | 503        |

**FIGURE  
2.2**

## A relational diagram



# Data Model – Building Blocks

- The basic building blocks of all data models are entities, attributes, relationships and constraints.
- **Entity**: person, place, thing or event about which data will be collected and stored. It is a distinguishable object
- **Attribute**: characteristic of an entity. For example: Name, Address and ContactNo for a person Entity.
- **Relationship**: describes an association among entities. Relationships are described in terms of degree, connectivity and existence
- **Constraint**: a restriction placed on the data

# Some definitions

- **Schema:** conceptual organization of the entire database. It includes the organization of data, the relationships between data elements, and the constraints that govern the data. In essence, it provides a high-level view of how data is organized and related within a database. It has following components
  - **Entities:** Represent tables or collections in a database.
  - **Attributes:** Represent fields or properties of entities.
  - **Relationships:** Indicate how different entities are related to each other.
- **Subschema:** conceptual organization of the portion of the database seen by the application programs that actually produce the desired information from the data within the database

# Example - Schema

- CREATE TABLE **Students** (
  - StudentID INT PRIMARY KEY,
  - FirstName VARCHAR(50),
  - LastName VARCHAR(50),
  - Age INT
- );
- CREATE TABLE **Courses** (
  - CourseID INT PRIMARY KEY,
  - CourseName VARCHAR(100),
  - Credits INT
- );
- CREATE TABLE **Enrollments** (
  - EnrollmentID INT PRIMARY KEY,
  - StudentID INT FOREIGN KEY REFERENCES Students(StudentID),
  - CourseID INT FOREIGN KEY REFERENCES Courses(CourseID),
  - Grade VARCHAR(2)
- );

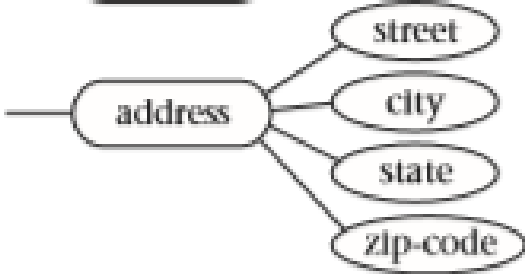
# Example – Sub Schema

## Sub-Schema for Student Records

```
❑ CREATE VIEW StudentRecords AS SELECT  Students.StudentID,  
    Students.FirstName,  Students.LastName,  Students.Age,  
    Enrollments.CourseID,  Enrollments.Grade FROM  Students  
    INNER JOIN Enrollments ON Students.StudentID =  
    Enrollments.StudentID;
```

# Some definitions

- **ER – Model:** The **entity-relationship** (*ER*) *data model* allows us to describe the data involved in a real-world enterprise in terms of objects and their relationships and is widely used to develop an initial database design (conceptual design).
- **Data Manipulation Language (DML):** Language that supports insert, update and delete operations in a database.
- **Data Definition Language (DDL):** Language that supports Create, Alter and Drop operations.

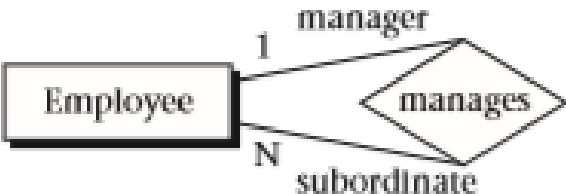
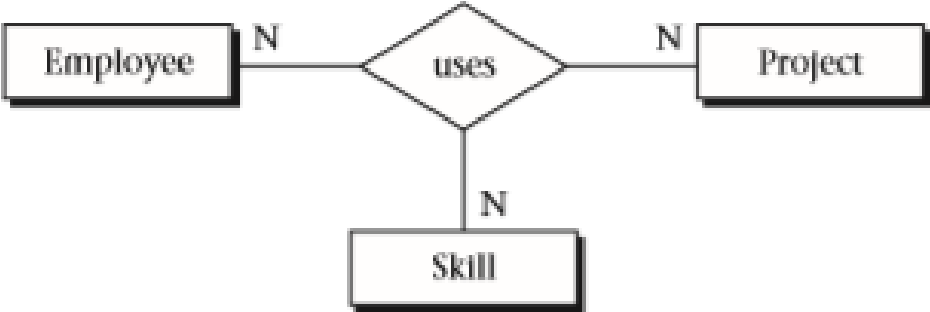
| <i>Concept</i>         | <i>Representation &amp; Example</i>                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------|
| Entity                 |  Employee            |
| Weak entity            |  Employee-job-history |
| Relationship           |  works-in            |
| Attribute              |                                                                                                         |
| Identifier (key)       |  emp-id               |
| descriptor (nonkey)    |  emp-name             |
| multivalued descriptor |  degrees              |
| complex attribute      |                      |



# A sample ER-Model

1. Teacher teaches multiple classes
2. A class consists of multiple students
3. There are many departments in a university but a teacher belongs to a particular department
4. A student can be enrolled in a particular program

# Degree of a Relationship

| <i>Concept</i>      | <i>Representation &amp; Example</i>                                                                                                                                                                                                                                                                            |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Degree</b>       |                                                                                                                                                                                                                                                                                                                |
| recursive<br>binary |  <p>Diagram illustrating a recursive binary relationship: <b>manages</b>. The relationship is between <b>Employee</b> entities. One <b>Employee</b> is the <b>manager</b> (1) and another is the <b>subordinate</b> (N).</p> |
| binary              |  <p>Diagram illustrating a binary relationship: <b>is-subunit-of</b>. The relationship is between <b>Department</b> (N) and <b>Division</b> (1).</p>                                                                         |
| ternary             |  <p>Diagram illustrating a ternary relationship: <b>uses</b>. The relationship is between <b>Employee</b> (N), <b>Project</b> (N), and <b>Skill</b> (N).</p>                                                                |

# Connectivity of a Relationship

## Connectivity

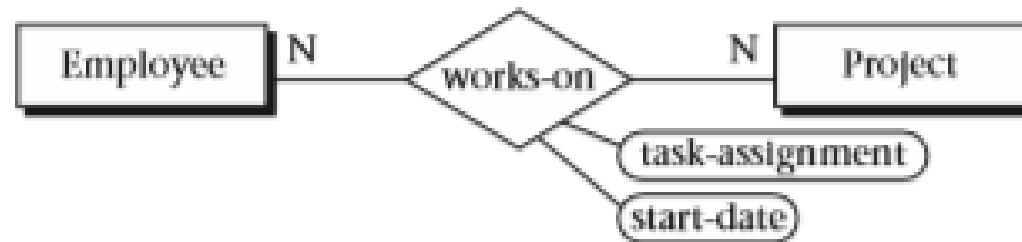
one-to-one



one-to-many



many-to-many



# Existence of a Relationship

## *Existence*

optional

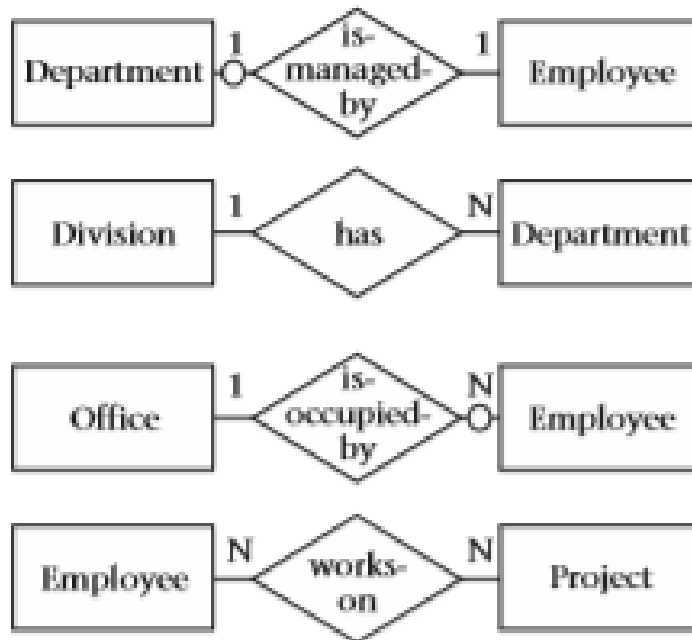


mandatory



# Alternative data-modeling notations

**ER model constructs using the Chen notation**



**ER model constructs using the "crow's-foot" approach [Knowledgeware]**

